

Apache Spark

Some tips on how to optimize it

2025/2026

1 Case Study Introduction: Initial IMDb Query

A naive approach to finding the top 10 highest-rated movies per genre in the 2010s looks like this:

```
from pyspark.sql import functions as F, Window

titles_df = spark.read.option("sep", "\t").csv("title.basics.tsv.gz", header=True)
ratings_df = spark.read.option("sep", "\t").csv("title.ratings.tsv.gz", header=True)

all_movies = titles_df.join(ratings_df, "tconst")
filtered = all_movies.filter((F.col("startYear") >= 2010) & (F.col("numVotes") >= 10000))
filtered = filtered.withColumn("genre", F.explode(F.split(F.col("genres"), ",")))

window = Window.partitionBy("genre").orderBy(F.desc("averageRating"), F.desc("numVotes"))
top10 = filtered.withColumn("rank", F.row_number().over(window)).filter(F.col("rank") <= 10)
top10.show()
```

This code loads two IMDb datasets, joins them on the movie ID (`tconst`), filters by year and popularity, and then finds the top 10 movies per genre using a window function. However, this approach is unoptimized and can be inefficient on large datasets. Before we apply improvements, we will analyze how Spark executes this code using its execution plan.

2 Understanding Spark Execution Plans

Before optimizing, it is helpful to know how to interpret Spark's execution plan for a query. This helps identify bottlenecks, expensive operations, and how Spark physically executes the code.

Using `df.explain(True)` or `df.explain("extended")` will show both the logical and physical plans:

```
df.explain(True) # Extended execution plan or
df.explain("extended")
```

Look for the following elements:

- **Physical operators** (e.g., Filter, Join, Aggregate, Exchange).
- **Data source scans** (ensure unnecessary columns are not read).
- **Shuffle operations** (reduce shuffle exchanges where possible).
- **Join types** (prefer BroadcastHashJoin over SortMergeJoin when feasible).

In detail:

- **Physical Operators:** In the planned output, you will see stages such as `Filter`, `Project`, `HashAggregate`, `Join`, `Exchange`, etc. Each line corresponds to an execution step. For example:

```
*(1) Filter (isnotnull(year#) && year# > 2010)
```

indicates that a filter is applied to the `year` column. The asterisk `*(1)` denotes that it is part of a whole-stage codegen pipeline.

- **Data Source Scans:** Look for lines with `Scan` or `FileScan`. These lines often show which columns were read and any pushed filters. For instance:

```
*(1) FileScan parquet [tconst, primaryTitle, startYear, genres]
PushedFilters: [IsNotNull(startYear), GreaterThan(startYear, 2010)]
```

This means Spark is reading only those columns and filtering data during the file scan itself. This is an efficient data access pattern.

- **Exchange and Join Types:** An `Exchange` represents a shuffle. For example:

```
Exchange hashpartitioning(genre#, 200)
```

means data is being repartitioned across the network. For joins, Spark might show:

```
BroadcastHashJoin [tconst], [tconst], BuildRight
```

or:

```
SortMergeJoin [tconst], [tconst]
```

The first is preferred when one side is small enough to broadcast. The second involves a shuffle and sort on both sides, which is more expensive.

- **WholeStageCodegen:** If you see lines prefixed by `*(2)` or similar, it indicates that Spark is fusing several physical operators into one compiled code stage. This improves CPU efficiency by avoiding function calls.
- **Stages vs Tasks:** Though not shown in the plan itself, remember that Spark jobs are divided into stages, separated by shuffles. Each stage runs in parallel tasks. You can view these in Spark's web UI to see per-stage metrics such as shuffle read/write, task execution time, and more.

In the IMDb query from the previous section, Spark is likely to:

- Perform a **SortMergeJoin** on the `full titles_df` and `ratings_df`, involving a shuffle.
- Scan all columns of the datasets, even those not used downstream.
- Shuffle data again during the `explode` and `window` operations.

This highlights opportunities for optimization, such as pushing filters earlier, selecting fewer columns, broadcasting smaller tables, and avoiding unnecessary shuffles.

3 Common Spark Performance Optimizations

Now that we have examined the execution plan of our initial query, let's explore key Spark optimizations and how they apply to improving performance. We will tie each technique back to our IMDb use case.

3.1 Partitioning and Data Pruning

Partitioning controls how data is split across the cluster. Good partitioning ensures even workload distribution and allows Spark to skip irrelevant partitions via pruning. For example, Parquet files partitioned by year can be pruned efficiently if the query filters by year. In our IMDb case, if the titles dataset were stored in a columnar format with partitioning, a filter like `startYear >= 2010` would avoid reading older data.

This works best with formats like Parquet and ORC, which support predicate pushdown and metadata filtering.

Summary:

If data is partitioned by year and stored in Parquet, Spark can skip irrelevant files:

```
df = spark.read.parquet("titles.parquet").filter("startYear >= 2010")
```

3.2 Caching and Persistence

If a DataFrame is reused across multiple operations, caching prevents recomputation. This is useful for iterative algorithms or multi-step queries.

In our example, if `filtered` is reused for further analysis (e.g., computing top movies, averages, genres), caching it avoids repeated filtering and joins.

Use:

```
df.cache() # Stores in memory (if possible)
df.persist(StorageLevel.MEMORY_AND_DISK) # Fallback to disk if memory is insufficient
```

Spark also compresses and column-prunes cached DataFrames internally.

Summary:

Cache when a DataFrame is used multiple times. Use:

```
df.cache() # memory
df.persist(MEMORY_AND_DISK) # memory + disk fallback
```

3.3 Broadcasting Small Datasets

If one side of a join is small (e.g., under 10MB), Spark can broadcast it to all executors. This avoids shuffling the larger table, making joins faster.

In our case, the `ratings_df` (after filtering popular titles) is small enough to broadcast. This turns a shuffle-heavy `SortMergeJoin` into a local `BroadcastHashJoin`.

To enforce this:

```
from pyspark.sql.functions import broadcast
optimized_df = large_df.join(broadcast(small_df), "key")
```

Summary:

Broadcast small tables to avoid shuffle joins:

```
optimized_df = movies_df.join(broadcast(ratings_df), "tconst")
```

3.4 Predicate Pushdown and Column Pruning

Spark's Catalyst optimizer pushes filter operations ("predicates") to the data source whenever possible. With columnar formats, it will: - Push filters to the scan level - Read only the selected columns

In our IMDb example:

```
df = spark.read.parquet("movies.parquet").filter("startYear >= 2010")
```

ensures that the scan will skip irrelevant data early.

This technique is also effective when reading from external databases using JDBC.

Summary:

Spark will skip rows and columns early if filters are applied as soon as possible.

3.5 Reducing Shuffles

Shuffles (data redistribution between partitions) are among the most expensive operations in Spark. They occur during joins, groupBy, and sorts.

In our IMDb example: - The join between titles and ratings causes a shuffle - Exploding genres and using window functions may trigger more shuffles

We can reduce shuffle cost by:

- Broadcasting small tables.
- Partitioning on join keys.
- Using reduceByKey instead of groupByKey when aggregating.

Summary:

Minimize shuffle cost by:

- Using reduceByKey over groupByKey.
- Broadcasting small join tables.
- Partitioning data on join/aggregation keys.

3.6 Using Built-in Functions over UDFs

Built-in DataFrame functions (e.g., from `pyspark.sql.functions`) are optimized in Spark's Catalyst engine and support `WholeStageCodegen`, which compiles them to efficient JVM bytecode.

Python UDFs, on the other hand, do not benefit from Spark optimizations and operate row-by-row, incurring serialization overhead.

In the IMDb query, use built-ins like:

```
F.split(F.col("genres"), ",")
```

rather than custom UDFs for string manipulation or transformation.

If UDFs are absolutely necessary, prefer Pandas UDFs which operate on column batches.

Summary:

Avoid Python UDFs. Use built-ins or vectorized UDFs:

```
# Good
df.withColumn("genre", F.explode(F.split("genres", ",")))
```

```
# Avoid
df.withColumn("genre", my_udf(df.genres))
```

4 Final Optimized Query

Now that we've examined the inefficiencies in the original IMDb query and reviewed relevant optimization techniques, let's put them into practice.

Below is the optimized version of the query, applying early filtering, column pruning, a broadcast join, and tuning Spark's configuration.

```
from pyspark.sql import functions as F, Window
from pyspark.sql.functions import broadcast

# Load and filter early
titles_df = spark.read.option("sep", "\t").csv("title.basics.tsv.gz", header=True)
ratings_df = spark.read.option("sep", "\t").csv("title.ratings.tsv.gz", header=True)

movies_2010s = titles_df.filter((F.col("startYear") >= 2010) & (F.col("titleType") == "
    movie")) \
    .select("tconst", "primaryTitle", "startYear", "genres")

popular_ratings = ratings_df.filter(F.col("numVotes") >= 10000) \
    .select("tconst", "averageRating", "numVotes")

# Use broadcast join
optimized_df = movies_2010s.join(broadcast(popular_ratings), "tconst")

# Explode genres and rank within each genre
optimized_df = optimized_df.withColumn("genre", F.explode(F.split("genres", ",")))

window = Window.partitionBy("genre").orderBy(F.desc("averageRating"), F.desc("numVotes"))

top10_genre = optimized_df.withColumn("rank", F.row_number().over(window)) \
    .filter(F.col("rank") <= 10)

top10_genre.show()
```

Tuning Spark Configuration

You can also adjust Spark configuration parameters to improve performance, especially when working with large datasets or running in a distributed environment:

```
spark.conf.set("spark.sql.shuffle.partitions", 100) # Better parallelism
spark.conf.set("spark.sql.adaptive.enabled", True) # Enable adaptive execution
```

These settings:

- Reduce the default number of shuffle partitions (from 200) to a more efficient value
- Enable Spark to dynamically adjust join strategies and shuffle partitions based on run-time statistics

5 Conclusion

By applying Spark optimization techniques such as:

- Filtering early and selecting only necessary columns,
- Broadcasting small datasets to avoid shuffles,
- Using built-in functions instead of Python UDFs,
- Partitioning and caching where appropriate,
- Tuning Spark configuration parameters,

we significantly improved the performance of a real-world Spark job. Starting with an unoptimized query, we used Spark's execution plan to identify inefficiencies, then incrementally applied improvements. Our optimized version achieved:

- Reduced data read from disk (through column and predicate pruning),
- Minimized network traffic (via broadcast joins and reduced shuffles),
- Faster execution (thanks to whole-stage code generation and better parallelism).

Understanding and analyzing the execution plan is crucial for effective optimization. Using tools like `df.explain(True)` and the Spark UI allows you to diagnose performance issues and understand the impact of each transformation.

Spark's power lies in its flexibility, but it's also easy to write inefficient code. By following these tips and constantly monitoring your jobs, you can ensure scalability and performance in large-scale data processing tasks.